CPGE PTSI-PT Lycée Jean Zay - Thiers

INFORMATIQUE

INTELLIGENCE ARTIFICIELLE

Cours

v0.3

Lycée Jean Zay – 21 rue Jean Zay – 63300 Thiers – Académie de Clermont-Ferrand



Compétences visées :

A3-08	Analyser les principes d'intelligence artificielle.
A4-03	Interpréter et vérifier la cohérence des résultats obtenus expérimentalement, analytiquement ou numériquement.
C1-03	Choisir une démarche de résolution d'un problème d'ingénierie numérique ou d'intelligence artificielle.
C3-03	Résoudre un problème en utilisant une solution d'intelligence artificielle.

Table des matières

1 L'Intelligence Artificielle, qu'est ce que c'est, à quoi ça sert ? 3

1.1 Premières définitions : IA, données . . . 3

1.2 Apprentissage automatisé (Machine Learning) . . . 4

1.3 Encore des définitions... . . . 7

2 Mécanismes d'apprentissages 8

2.1 Classification des algorithmes d'apprentissage . . . 8

2.2 Validation du modèle . . . 8

2.2.1 Critères de validation des problèmes de classification . . . 8

2.2.2 Critères de validation des problèmes de régression . . . 9

2.3 Séparation des données/gestion des données . . . 9

2.3.1 Types de données – Apprentissage supervisé . . . 10

2.3.2 Normalisation des données . . . 10

2.3.3 Séparation des données . . . 10

3 Algorithmes d'apprentissage 11

3.1 Algorithme des k plus proches voisins . . . 11

3.1.1 Présentation . . . 11

3.1.2 L'algorithme . . . 12

3.2 Régression univariée (Chloé-Agathe Azencott) . . . 13

4 Le neurone, les réseaux de neurones 14

4.1 Introduction aux réseaux de neurones . . . 14

4.1.1 Bref historique . . . 14

4.1.2 Exemples d'applications des réseaux de neurones . . . 15

4.2 Modèle de neurone . . . 15

4.2.1 Neurone (ou perceptron) . . . 15

4.2.2 Fonctions d'activation . . . 16

4.3 Réseaux de neurones . . . 17

4.3.1 Modélisation d'un réseau de neurones . . . 17

4.3.2 Fonction de coût . . . 20

4.3.3 Notion de rétropropagation – Descente de gradient . . . 20

4.3.4 Fin d'apprentissage . . . 22

4.4 Pour aller plus loin... . . . 23

1 L'Intelligence Artificielle, qu'est ce que c'est, à quoi ça sert ?

1.1 Premières définitions : IA, données

Définition *Intelligence Artificielle*

L'intelligence artificielle (IA) est « l'ensemble des théories et des techniques mises en œuvre en vue de réaliser des machines capables de simuler l'intelligence » (source : Wikipedia).

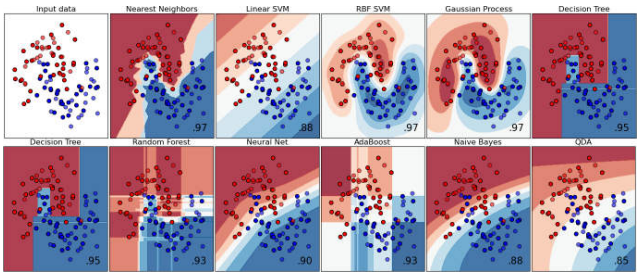
En première approche, l'IA peut être assimilée à un ensemble d'algorithmes permettant de réaliser des tris, des classifications.

Exemples :



Source : https://learnopencv.com

(a) Identification d'objets, en temps réel, dans une vidéo.



Source : https://scikit-learn.org/

(b) Comparaison du résultats d'algorithmes de classification permettant de classifier un nuage de points bleus et rouges de transparences différentes.

FIGURE 1 – Exemples d'application de l'Intelligence Artificielle

On remarquera sur l'exemple de gauche que l'algorithme parvient à identifier sur l'image des zones puis à identifier à quoi elles correspondent. On remarquera aussi que la flamme est identifiée comme étant un donut...

Pour utiliser un algorithme d'IA il est nécessaire de disposer de données... en général une très grande quantité de données.

Définition *Données*

Les données utilisées par un algorithme d'IA peuvent être sous différentes formes :

- des données quantitatives continues qui peuvent prendre n'importe quelles valeurs dans un ensemble de valeurs (par exemple la température) ;
- des données quantitatives discrètes qui peuvent prendre un nombre limité de valeurs dans un ensemble de valeurs (par exemple nombre de pièces d'un logement) ;
- des données qualitatives (ordinales ou nominales suivant qu'on puissent les classer ou non, par exemple des couleurs, des notes à un test d'opinion ...).

Pour pouvoir traiter ces données, il peut être nécessaire qu'elles soient organisées sous une certaine forme. On peut par exemple identifier :

- les données structurées, dans une base de données par exemple ;
- les données semi-structurées, dans un fichier `csv`, `xml` ou `json` par exemple ;
- les données non structurées comme un image, du texte, ou une vidéo.

Exemples :

- Le site https://www.kaggle.com/ partage des sets de données.
- On peut par exemple retrouver un dataset semi-structuré au format `json` ou `csv` des chansons disponibles sur *Spotify*, sorties entre 1922 et 2020 (https://www.kaggle.com/spotify-da[...]tracks).
- Le site https://storage.googleapis.com/ propose des sets de photos étiquetées selon 600 classes.

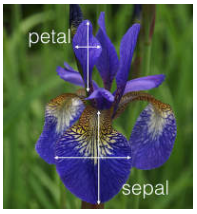
Définition *Observations – Caractéristiques*

On appelle observation (ou individu ou objet) une « ligne de donnée » qui va être utilisée par un algorithme d'apprentissage.

Une observation est composée de caractéristiques qui varient pour chacun des observations.

Exemple :

Dans le cas de la base de donnée des iris, présente par exemple dans `scikit-learn`, les données sont composées de 150 observations, chacune composée de 4 caractéristiques : longueur et largeur du sépale ainsi que longueur et largeur du pétale.



Définition *Big data*

Mégadonnées ou données massives. Le big data désigne les ressources d'informations dont les caractéristiques en termes de volume, de vélocité et de variété imposent l'utilisation de technologies et de méthodes analytiques particulières pour générer de la valeur, et qui dépassent en général les capacités d'une seule et unique machine et nécessitent des traitements parallélisés (source : Wikipedia).

Que fait-on avec ces données ? En général les algorithmes vont chercher un lien entre ces données. Dans le cas où il existe un lien connu par le *Data Scientist* on parle d'apprentissage supervisé.

Si ce lien n'est pas connu, on parle d'apprentissage non supervisé ou de clustering.

1.2 Apprentissage automatisé (Machine Learning)

Définition *Apprentissage automatique (Machine learning)*

L'apprentissage automatique est un champ de l'intelligence artificielle dont l'objectif est d'analyser un grand volume de données afin de déterminer des motifs et de réaliser un modèle prédictif.

L'apprentissage comprend deux phases :

- l'entraînement (ou apprentissage) est une phase d'estimation du modèle à partir de données d'observations ;
- la mise en production du modèle (inférence) est une phase pendant laquelle de nouvelles données sont traitées dans le but d'obtenir le résultat souhaité.

L'entraînement peut être poursuivi même en phase de production.

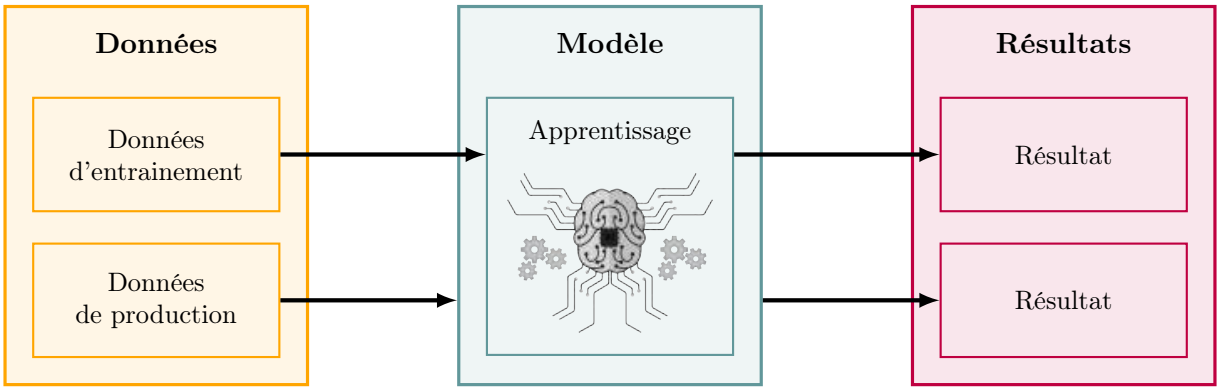


FIGURE 2 – Apprentissage automatique (Machine Learning)

Définition *Apprentissage supervisé – Apprentissage*

Tâche d'apprentissage au cours duquel l'algorithme (ou fonction de prédiction) va, à partir d'un ensemble de données **étiquetées**, déterminer un lien entre un ensemble les données et les étiquettes.

Définition *Apprentissage supervisé – Inférence*

Tâche au cours duquel l'algorithme (ou fonction de prédiction) va, à partir d'un ensemble de données **non étiquetées**, prédire l'étiquette.

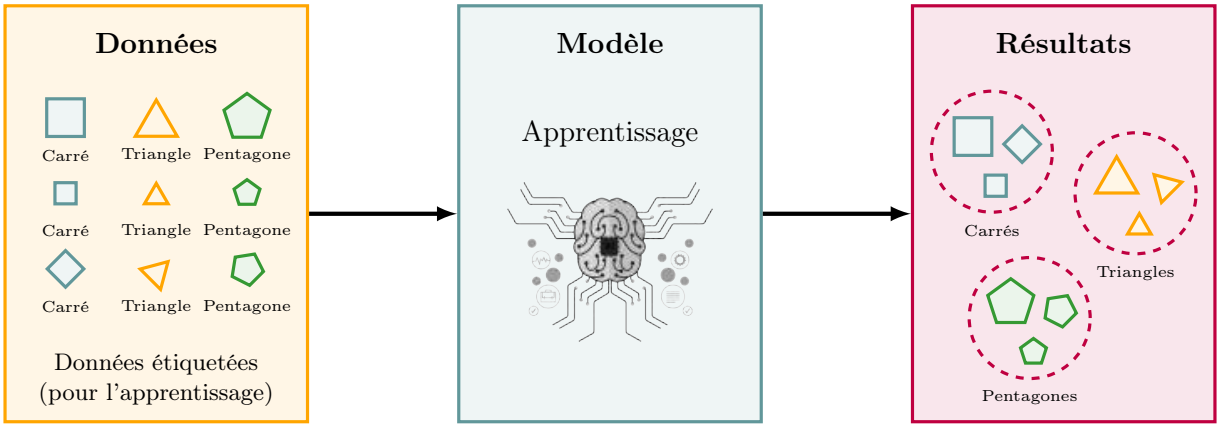


FIGURE 3 – Apprentissage supervisé

Exemple :

Soit un ensemble d'images en noir et blanc représentant des chiffres de 0 à 9. L'algorithme doit dans un premier temps *apprendre* le lien entre l'image et le chiffre.

Dans un second temps, l'algorithme devra déterminer le chiffre en fonction d'une image seule.

Définition *Apprentissage non supervisé – Clustering*

Tâche d'apprentissage au cours duquel l'algorithme (ou fonction de prédiction) va, à partir d'un ensemble de données **non étiquetées**, déterminer un lien entre les données (et les regrouper).

Exemple :

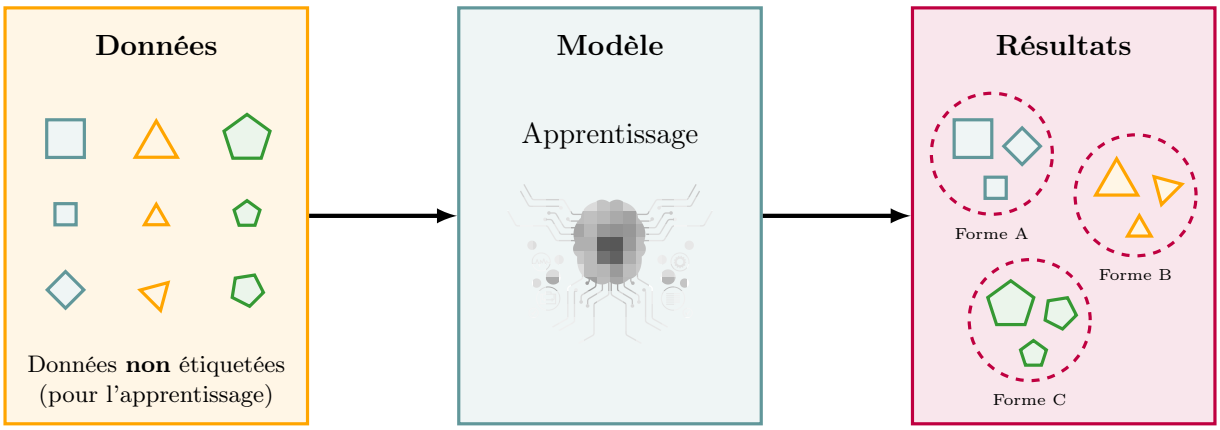


FIGURE 4 – Apprentissage non supervisé

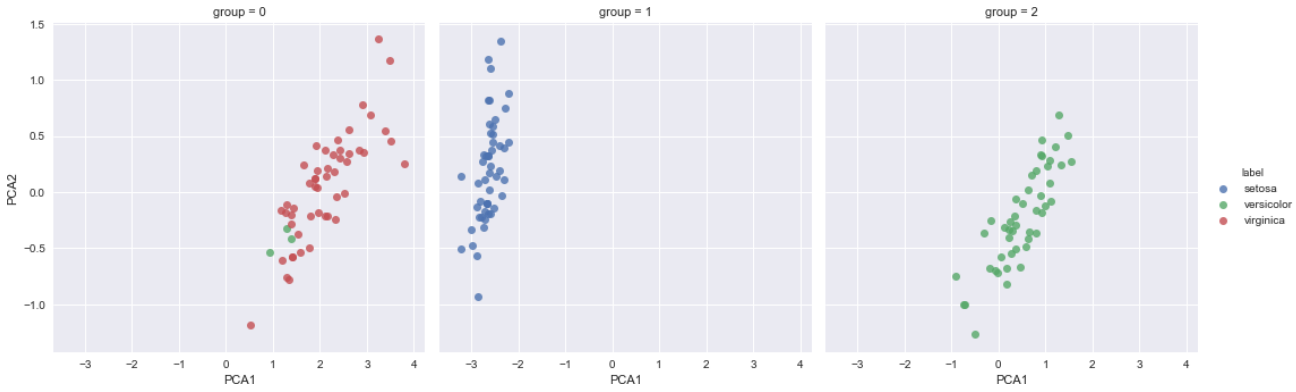


FIGURE 5 – Classification d'iris en utilisant un algorithme d'apprentissage non supervisé

On remarque (lorsque l'image est en couleur) que sur des mesures sur 150 iris, l'algorithme a réussi à retrouver les 3 différentes espèces, sans les connaître, à 3 erreurs près.

Définition *Apprentissage par renforcement*

Si au cours de l'apprentissage supervisé, un mécanisme de récompense est mis en œuvre pour améliorer les performances du modèle, on parle d'apprentissage par renforcement.

On peut maintenant faire une synthèse des méthodes d'apprentissage dont nous avons pris connaissance (incomplète, car il en existe d'autres) :

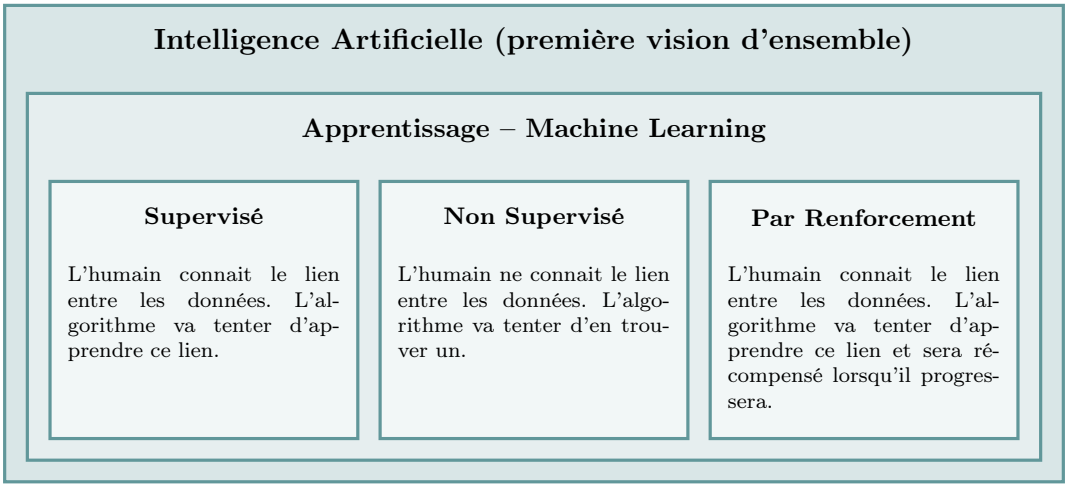


FIGURE 6 – Intelligence Artificielle : première vision d'ensemble

1.3 Encore des définitions...

Définition *Classification*

En apprentissage automatique, on appelle problème de classification les problèmes pour lesquels les étiquettes sont discrètes.

Exemple :

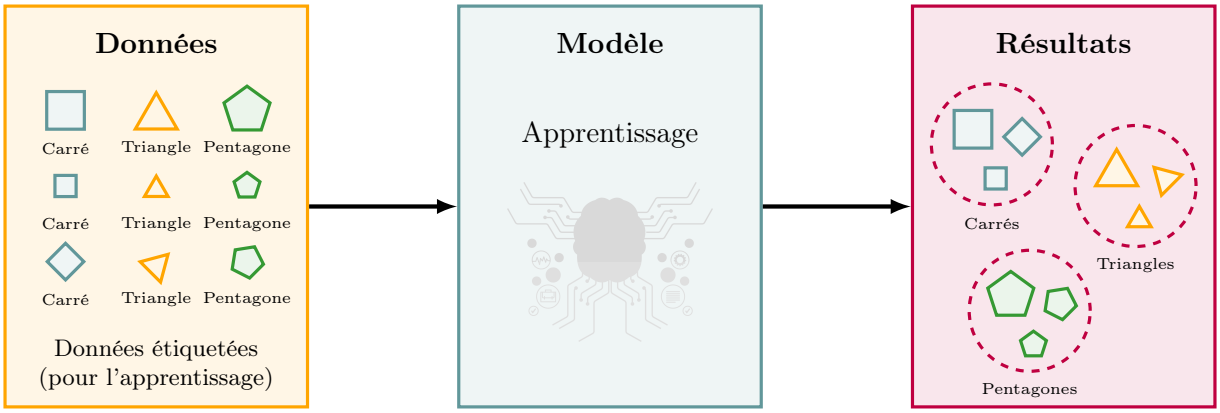


FIGURE 7 – Classification

Dans le cas où nous souhaitons regrouper par forme, il s'agit d'un problème de classification. Les classes discrètes sont les formes (triangles, carrés, pentagones ...).

Il serait aussi possible d'opérer un regroupement par couleur. Les classes seraient donc différentes.

Définition *Régression*

En apprentissage automatique, on appelle problème de régression les problèmes pour lesquels les étiquettes sont continues.

Exemple :

Dans le cas où nous souhaiterions que l'algorithme détermine l'aire ou le périmètre de la forme, les étiquette seraient alors continues sur $\mathbb{R}_*^+$.

2 Mécanismes d'apprentissages

2.1 Classification des algorithmes d'apprentissage

La bibliothèque `scikit-learn` propose une classification de divers algorithmes en fonction des apprentissages automatiques (hors réseaux de neurones).

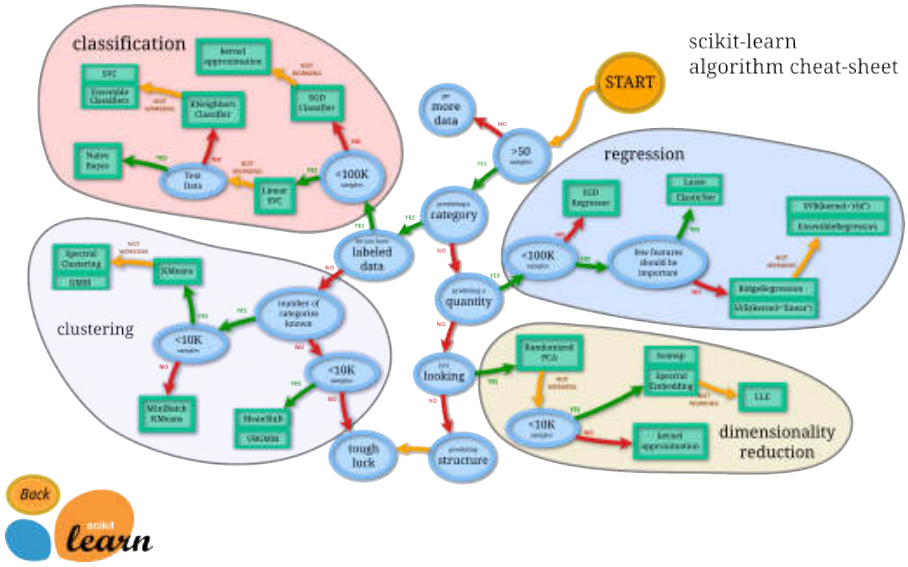


FIGURE 8 – La bibliothèque `scikit-learn`

2.2 Validation du modèle

Une fois un algorithme d'apprentissage choisi, on se pose la question de la validation du modèle. Quels critères et outils vont nous permettre de considérer que notre apprentissage est « bon » ?

Lors d'un problème de classification, il est par exemple possible de déterminer les écarts entre les valeurs prédites par l'algorithme et les valeurs cibles.

2.2.1 Critères de validation des problèmes de classification

Définition *Valeur prédictive positive*

Valeur prédictive positive (*accuracy classification score*) :

$$\text{Justesse} = \frac{\text{Nombre de prédictions vraies}}{\text{Nombre de prédictions totales}}$$

```python
import sklearn.metrics as skm

# X(numpy.ndarray) : données d'entrées
# Y(numpy.ndarray) : données cibles correspondantes
# Y_pred(numpy.ndarray) : données prédites par l'algorithme de classification

print skm.accuracy_score(Y, Y_pred)
```

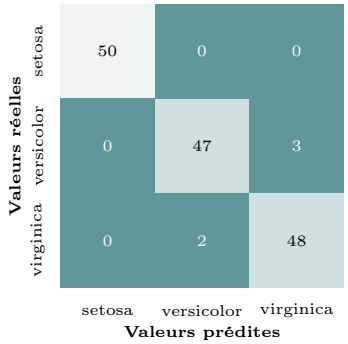
Définition *Matrices de confusion*

La matrice de confusion est une métrique permettant de déterminer la qualité d'une classification. En abscisses sont indiquées le valeurs réelles, et on ordonnée les valeurs prédites.

Exemple :

Dans l'exemple ci-contre, nous cherchons à classifier des iris, grâce à un algorithme, selon 3 familles : les setosa, les versicolor et les virginica.

Sur la diagonale, sont retrouvées les iris dont l'espèce a été correctement prédite. En revanche, 3 versicolor ont été classées parmi les virginica et 2 virginica ont été classifiées dans les versicolor.



Remarque

La matrice de confusion présentée est dite non-normalisée. Si la répartition des étiquettes n'est pas uniforme (en fonction des classes), il est probable qu'il y ait davantage d'erreurs pour les classes ayant beaucoup d'étiquettes. Pour palier ce problème on peut utiliser des matrices de confusions normalisées (on divise alors chaque terme de la matrice par la somme des éléments de la même ligne).

2.2.2 Critères de validation des problèmes de régression

Définition *Erreur moyenne quadratique*

Appelée également *mean squared error*, il s'agit d'une moyenne d'écart au carré :

$$\text{msq} = \frac{1}{N}\sum_{i=1}^{n}\left(Y_i - \hat{Y}_i\right)^2$$

en notant Y_i la valeur réelle (étiquetée) et $\hat{Y}_i$ la valeur estimée par l'algorithme.

2.3 Séparation des données/gestion des données

Lorsqu'on souhaite disposer d'un modèle défini par un algorithme d'IA, il faut en premier lieu disposer de données !

2.3.1 Types de données – Apprentissage supervisé

En apprentissage supervisé, il est nécessaire de connaître des données d'entrées ainsi que les sorties correspondantes (appelées aussi **cibles** ou **target**).

Dans le cadre d'une programmation Python avec la bibliothèque `scikit-learn` les données d'entrée et de sortie peuvent être implémentées en utilisant des `numpy.ndarray`.

Ainsi, si les données d'entrées, stockées dans la variable `data` sont composées de n observations elles mêmes composées de p catégories, le `shape` de `data` sera `(n,p)`.

La donnée cible sera quant à elle un vecteur de n valeurs.

Exemple :

Données d'entrées : matrice `X` de n observations avec 4 caractéristiques.

Données de sortie : vecteur `Y` de n résultats.

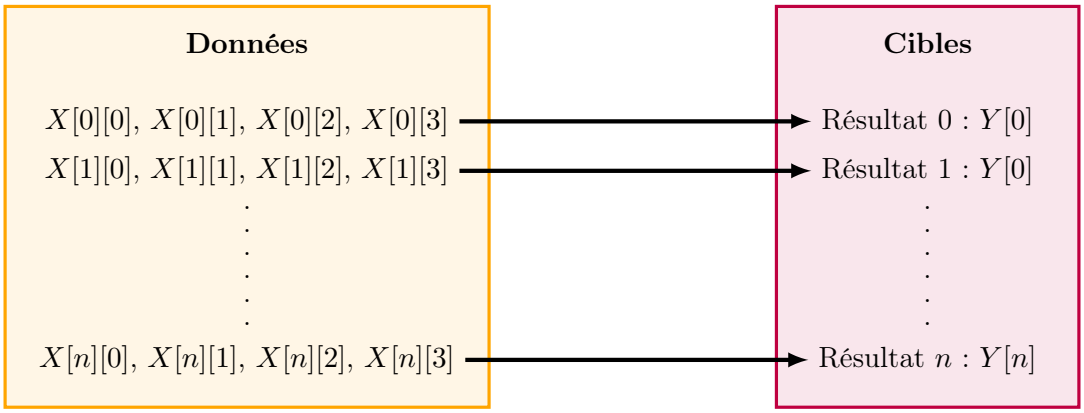


FIGURE 9 – Données d'entrée et de sortie

Dans le cadre d'une classification, où r résultats sont possibles. On peut attribuer une valeur (entière) entre 1 et r à chacun des résultats, notamment si ceux-ci sont des données qualitatives.

2.3.2 Normalisation des données

https://scikit-learn.org/stable/modules/preprocessing.html

2.3.3 Séparation des données

Lors du démarrage d'un apprentissage supervisé, on dispose d'un jeu de données comprenant les données d'entrée et les cibles correspondantes. Il est d'usage de séparer ces données en deux parties :

- les données d'entraînement permettant... d'entraîner le modèle. Dans la pratique on utilise entre 60 et 80 % des données de base ;
- les données de te test, permettant de valider l'apprentissage (ou le modèle), dans la pratique entre 20 et 40 % des données de base.

On commence donc par réaliser un apprentissage sur les données d'entraînement. Cet entraînement produit des résultats. Connaissant les cibles correspondant aux données d'entraînement, on peut donc en déduire une performance du modèle sur le traitement des données d'entraînement.

On utilise alors le modèle en lui donnant les données de test. De même, on peut donc en déduire une performance du modèle sur le traitement des données de test.

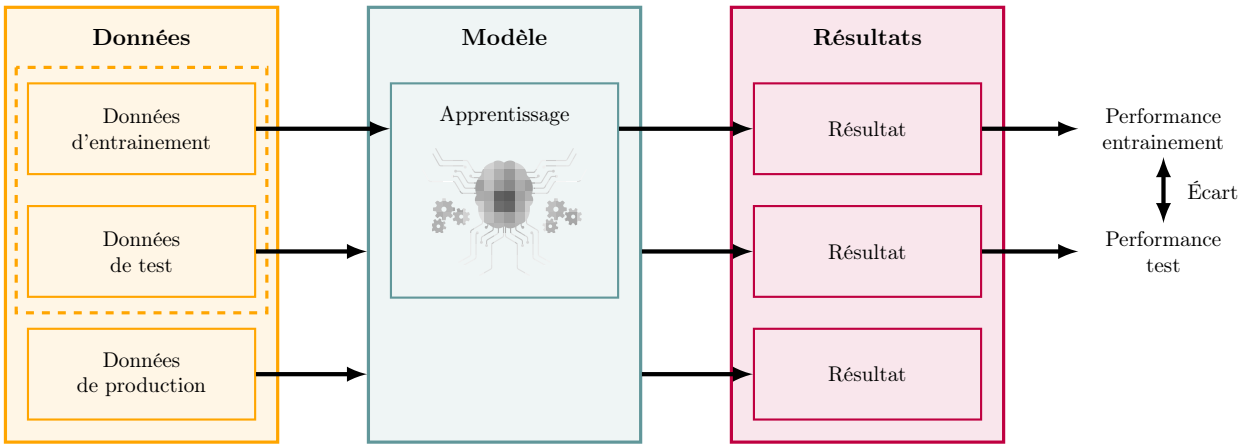


FIGURE 10 – Séparation des données

Se pose alors le problème de la méthode à utiliser pour séparer les données. En effet, les données de base pouvant potentiellement être triées (ordonnées par rapport à une des caractéristiques), en sélectionner une partie pourrait créer un biais dans l'apprentissage.

`scikit-learn` permet de séparer aléatoirement des données en fixant le pourcentage de données de validation.

```
from sklearn.model_selection import train_test_split
# X(numpy.ndarray) : données d'entrées
# Y(numpy.ndarray) : données cibles correspondantes
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.33)
```

On perçoit donc que la qualité de l'apprentissage peut dépendre du choix utilisé lors de la séparation des données. De plus, certains choix de paramètres permettant d'affiner le modèle sont aussi liés aux données d'entraînement sélectionnées.

Une des solutions pour résoudre ce problème est d'avoir recours à la validation croisée.

https://scikit-learn.org/stable/modules/cross_validation.html#cross-validation

3 Algorithmes d'apprentissage

3.1 Algorithme des k plus proches voisins

3.1.1 Présentation

L'algorithme des k plus proches voisins (aussi appelé k-nearest neighbors algorithm - KNN) permet de réaliser des opérations de régression et de classification sur un ensemble de données.

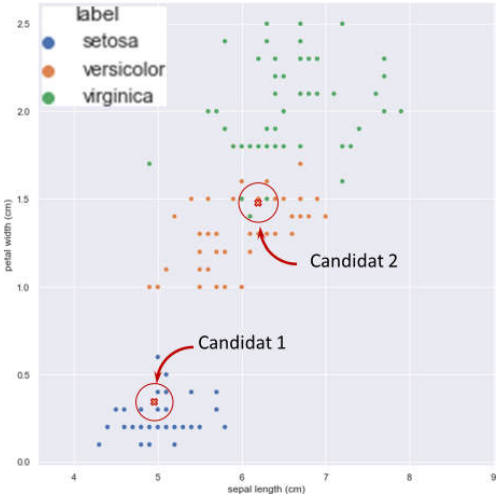
Pour une première approche, cette méthode permet d'estimer la classe d'une nouvelle donnée en déterminant la norme entre cette nouvelle donnée et l'ensemble des données du jeu d'entraînement. Parmi les k plus proches voisins, on recherche la classe majoritaire et on attribue cette classe à la nouvelle donnée.

Exemple :

Dans le cas ci-contre, on cherche à classifier les iris selon 3 catégories (setosa, versicolor et virginca). Les données étiquetées sont représentées dans le plan longueur du sépale en abscisse et largeur du pétale en ordonnée.

Soient les candidats 1 et 2, deux iris dont ont connaît les caractéristiques mais pas l'espèce. Les 5 plus proches voisins du candidat 1 sont des setosa. Il est donc probable que le candidat 1 soit un setosa aussi.

Concernant le candidat 2, parmi les 5 plus proches voisins, 3 sont des virginica et 2 sont des versicolor. On peut faire l'hypothèse que c'est un virginica.



3.1.2 L'algorithme

Prenons comme exemple des données sous la forme `[x,y,c]` où `x` et `y` représentent des coordonnées et `c` la classe d'appartenance.

Première étape – Calculer la distance

Il faut tout d'abord définir une fonction de distance. On peut par exemple utiliser la distance euclidienne. Pour deux vecteurs x_1 et x_2 de taille N :

$$d = \sqrt{\sum_{i=1}^{n} (x_{1,i} - x_{2,i})^2}$$

```
import math as m
def distance(x1:list, x2:list) -> float :
    distance = 0.
    for i in range(len(x1)):
        distance += (x1[i]-x2[i])**2
    return m.sqrt(distance)
```

Deuxième étape – Détermination des plus proches voisins

Pour cette étape, on dispose de données pour l'entraînement `data_train` et d'une donnée à tester `data_test`. Il va falloir calculer la distance entre la donnée à tester et les données d'entraînement puis trier les données. Enfin, on retournera les k lignes les plus proches de `data_test`.

```
def get_voisins(data_train:list, data_test, k:int) -> list :
    distances =[]
    for ligne in data_train :
        d = distance(ligne,data_test)
        distances.append([ligne,d])
    # Tri de la liste suivant la seconde colonne (distances)
    distances.sort(key=lambda t : t[1])
    voisins = []
    for i in range(k) :
        voisins.append(distance[i][0])
    return voisins
```

Troisième étape – Prédiction

Une fois qu'on connaît les k plus proches voisins, on regarde quelle est la classe majoritaire parmi ces voisins.

```python
def prediction(data_train:list, data_test, k:int) -> list :
    voisins = get_voisins(data_train, data_test, k)
    sorties = [ligne[-1] for ligne in voisins]
    predict = max(set(sorties), key=sorties.count)
    return predict
```

Prédiction avec `scikit-learn`

Un exemple rapide pour réaliser une classification KNN avec `scikit-learn`.

```python
import numpy as np
from sklearn import datasets
from sklearn.neighbors import KNeighborsClassifier

iris_X, iris_y = datasets.load_iris(return_X_y=True)

# Fait en sorte que np.random renvoie toujours les mêmes valeurs
np.random.seed(0)

# Chargement des données et séparation des données
indices = np.random.permutation(len(iris_X))
iris_X_train = iris_X[indices[:-10]]
iris_y_train = iris_y[indices[:-10]]
iris_X_test = iris_X[indices[-10:]]
iris_y_test = iris_y[indices[-10:]]

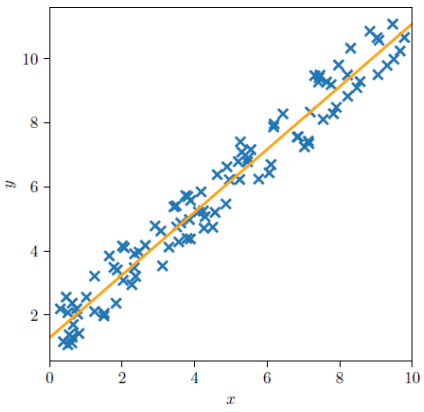
# Création du classifieur
knn = KNeighborsClassifier() # KNeighborsClassifier(n_neighbors=k) pour choisir k

# Entraînement du classifieur
knn.fit(iris_X_train, iris_y_train)

# Prédiction sur le jeu de test (à comparer avec iris_y_test)
knn.predict(iris_X_test)
```

3.2 Régression univariée (Chloé-Agathe Azencott)

Dans le cadre de la régression linéaire univariée, on dispose de n observations : $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), ..., (x_i, y_i), ..., (x_n, y_n)\}$ et des étiquettes y_i associées. La régression linéaire consiste à choisir une fonction de prédiction f de la forme, $\forall x \in \mathbb{R}, f(x) = ax + b$ avec$(a, b) \in \mathbb{R}^2$. Il va donc falloir minimiser l'erreur entre les prédictions et la base de données d'entraînement. La minimisation se note :



$$\left(\hat{a}, \hat{b}\right) = \arg \min_{(a,b)\in\mathbb{R}^2} \frac{1}{n} \sum_{i=1}^{n} (y_i - (ax_i + b))^2$$

(Méthode des moindres carrés)

On montre que si $\sum_{i=1}^{n} x_i^2 \neq \left(\sum_{i=1}^{n} x_i\right)^2$ le problème à une unique solution. Sinon le système est indéterminé et il y a une infinité de solutions.

Exemple de régression linéaire univariée avec `scikit-learn`

```python
import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets, linear_model
from sklearn.metrics import mean_squared_error, r2_score

# Load the diabetes dataset
diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)

# Use only one feature
diabetes_X = diabetes_X[:, np.newaxis, 2]

# Split the data into training/testing sets
diabetes_X_train = diabetes_X[:-20]
diabetes_X_test = diabetes_X[-20:]

# Split the targets into training/testing sets
diabetes_y_train = diabetes_y[:-20]
diabetes_y_test = diabetes_y[-20:]

# Create linear regression object
regr = linear_model.LinearRegression()

# Train the model using the training sets
regr.fit(diabetes_X_train, diabetes_y_train)

# Make predictions using the testing set
diabetes_y_pred = regr.predict(diabetes_X_test)

# The coefficients
print('Coefficients: \n', regr.coef_)
# The mean squared error
print('Mean squared error: %.2f' % mean_squared_error(diabetes_y_test, diabetes_y_pred))
# The coefficient of determination: 1 is perfect prediction
print('Coefficient of determination: %.2f' % r2_score(diabetes_y_test, diabetes_y_pred))

# Plot outputs
plt.scatter(diabetes_X_test, diabetes_y_test,  color='black')
plt.plot(diabetes_X_test, diabetes_y_pred, color='blue', linewidth=3)
plt.xticks(())
plt.yticks(())
plt.show()
```

4 Le neurone, les réseaux de neurones

4.1 Introduction aux réseaux de neurones

4.1.1 Bref historique

Dans les années 1940, les chercheurs tentent de fabriquer une machine capable d'apprendre à partir de données fournies, de mémoriser des informations et de traiter des informations incomplètes. Pour

cela, ils essayent de réaliser des modèles mathématiques de neurones biologiques. S'en suit alors la naissance des premiers modèles de perceptrons.

L'apprentissage automatisé va alors connaître des hauts et des bas, au gré des avancées scientifiques et technologiques. Dans les années 1960, un des premiers coups d'arrêt fût provoqué par la non-capacité des réseaux de neurones à traiter des problèmes non linéaires. Dans les années 1980, la rétropropagation du gradient fut proposée. Mais devant le manque de capacité des ordinateurs, la recherche marqua un second coup d'arrêt. Dans les années 1990/2000, l'apparition des réseaux convolutifs et leur capacité à analyser les données des images relançât alors les recherches dans ce domaine.

4.1.2 Exemples d'applications des réseaux de neurones

- Banque : prêts et scoring.
- Cartes de crédit : détection des fraudes.
- Finance : analyse d'investissements et de fluctuations des taux de change.
- Assurance : couverture assurantielle et estimation des réserves.
- Marketing : ciblage des prospections, mesures et comparaisons des campagnes et des méthodes.
- Archéologie : identification et datation de fossiles et d'ossements.
- Défense : identification de cibles.
- Environnement : prévisions de la qualité de l'air et de l'eau.
- Production : contrôles qualité.
- Médecine : diagnostics médicaux.
- Energies : estimations des réserves, prévisions de prix.
- Pharmacie : efficacité de nouveaux médicaments.
- Psychologie : prévisions comportementales.
- Immobilier : études de marchés.
- Recherche scientifique : identification de spécimens, séquençages de protéines.
- Télécommunication : détection des pannes de réseaux.
- Transport : maintenance des voies.

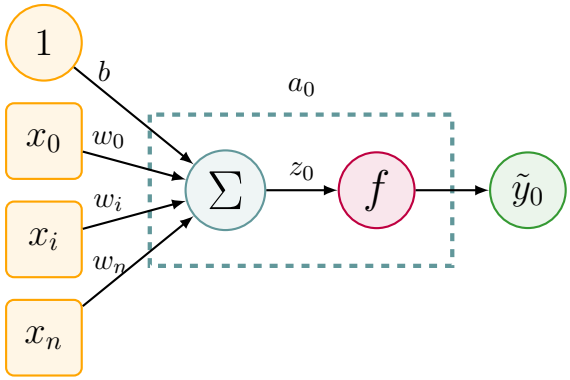
Source : https://fr.wikiversity.org/

4.2 Modèle de neurone

4.2.1 Neurone (ou perceptron)

Prenons la représentation ci-contre pour un neurone. On note :

- $\mathbf{X}$ le vecteur d'entrée constitué des données x_i. Ce vecteur constitue la couche d'entrée ;
- w_i les poids (poids synaptiques) ;
- b le biais ;
- z_0 la somme pondérée des entrées ;
- f une fonction d'activation ;
- $\tilde{y}_0$: la valeur de sortie du neurone.



On a donc, dans un premier temps :

$$z_0 = b + \sum_{i=0}^{n} w_i x_i$$

Après la fonction d'activation, on a donc en sortie du neurone :

$$\tilde{y}_0 = a_0 = f(z_0) = f\left(b + \sum_{i=0}^{n} w_i x_i\right)$$

Remarque

- La notation tilde ($\tilde{y}_0$) vient du fait que la valeur de sortie d'une neurone est une valeur estimée qu'il faudra comparer à y_0 valeur de l'étiquette utilisée pour l'apprentissage supervisé.
- Par la suite, dans la représentation graphique on ne fera apparaître ni la somme pondérée ni la fonction d'activation, mais seulement la valeur de sortie du neurone (notée par exemple a_0).

4.2.2 Fonctions d'activation

Définition *Fonction d'activation*

Les fonctions d'activation sont des fonctions mathématiques appliquées au signal de sortie (z). Il est alors possible d'ajouter des non linéarités à la somme pondérée.

On donne ci-dessous quelques fonctions usuelles :

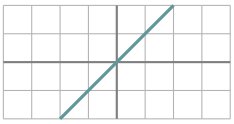
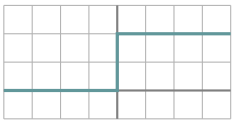
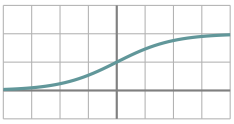
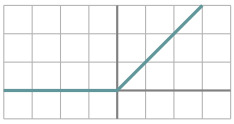
Identité	Heaviside	Logistique (sigmoïde)	Unité de rectification linéaire (ReLU)
			
$f(x) = x$	$f(x) = \begin{cases} 0 \text{ si } x < 0 \\ 1 \text{ si } x \geq 0 \end{cases}$	$f(x) = \frac{1}{1+\mathrm{e}^{-x}}$	$f(x) = \begin{cases} 0 \text{ si } x < 0 \\ x \text{ si } x \geq 0 \end{cases}$

FIGURE 11 – Fonctions d'activation usuelles

La FIGURE 12 montre l'influence des poids et des biais sur la sortie du perceptron en utilisant une fonction d'activation ReLU.

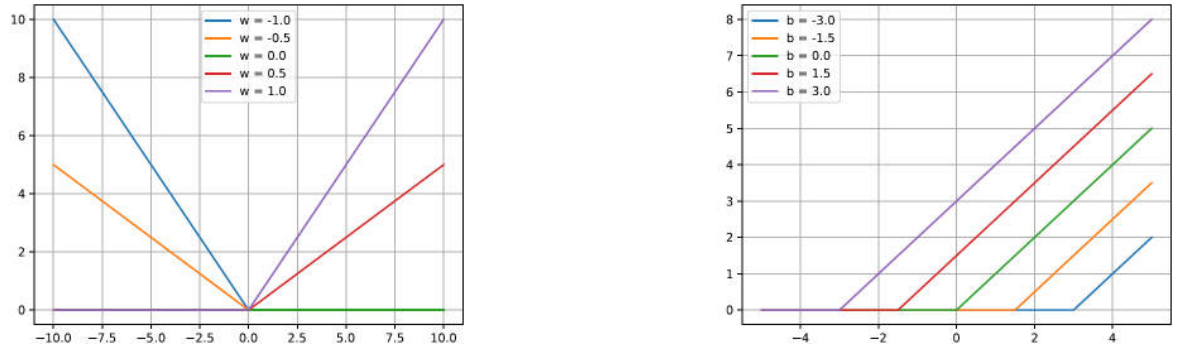


FIGURE 12 – Influence des poids et des biais sur la sortie du perceptron

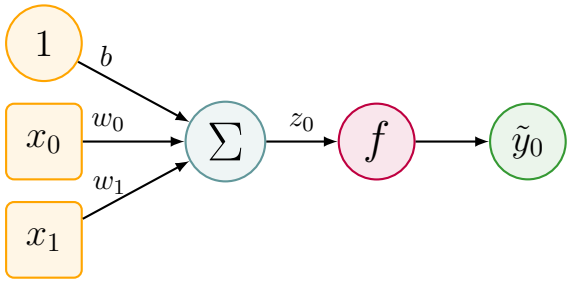
On peut ainsi voir qu'avec la fonction d'activation ReLU, plus le poids sera grand en valeur absolue, plus le neurone amplifiera le signal d'entrée.

Le biais permettra de prendre en compte le « niveau » du signal d'entrée à partir duquel, le signal doit être amplifié, ou non.

Exemple :

Prenons un neurone à deux entrées binaires. Initialisation les poids et le biais avec des valeurs aléatoires : $w_0 = -0,3$, $w_1 = 0,8$ et $b = 0,2$.

On peut donc évaluer l'ensemble des sorties calculable par le neurone.



x_0	x_1	z	Id.	H.	Sig.	ReLu
0	0	0,2	0,2	1	0,549	0,2
0	1	1	1	1	0,731	1
1	0	-0,1	-0.1	0	0,475	0
1	1	0,7	0.7	1	0,668	0,7

FIGURE 13 – Ensemble des sorties calculables par le neurone

4.3 Réseaux de neurones

Le site https://playground.tensorflow.org/ permet de visualiser et d'expérimenter un réseau de neurones.

4.3.1 Modélisation d'un réseau de neurones

Définition *Couches*

Un réseau de neurones est un ensemble de neurones reliés, par couches, entre eux.

Dans un réseau de neurones **dense** tous les neurones de la couche i sont reliés à tous les neurones de la couche $i + 1$.

On peut ainsi distinguer plusieurs types de couches :

- Couche d'entrée : cette couche est une copie de l'ensemble des données d'entrées. Le nombre de neurones de cette couche correspond donc aux nombre de données d'entrées. On note $\mathbf{X} = (x_0, ..., x_n)$ le vecteur d'entrées.
- Couche cachée (ou couche intermédiaire) : il s'agit d'une couche qui a une utilité intrinsèque au réseau de neurones. Ajouter des neurone dans cette couche (ou ces couches) permet donc d'ajouter de nouveaux paramètres. Pour une couche, la même fonction d'activation est utilisée pour tous les neurones. En revanche la fonction d'activation utilisée peut être différente pour deux couches différentes. Les fonctions d'activations des couches intermédiaires sont souvent non linéaires.
- Couche de sortie : le nombre de neurones de cette couche correspond au nombre de sorties attendues. La fonction d'activation de la couche de sortie est souvent linéaire. On note $\mathbf{Y} = (y_0, ..., y_p)$ le vecteur des sorties.

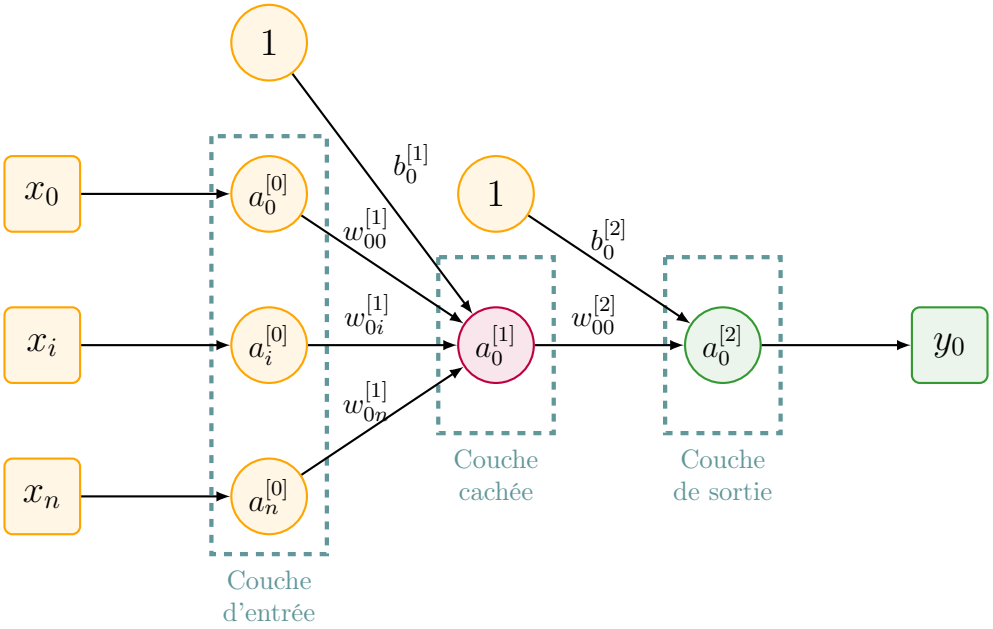


FIGURE 14 – Les différentes couches d'un réseau de neurones

En utilisant la loi de comportement du modèle de perceptron, on peut donc exprimer $\mathbf{Y} = \mathcal{F}(\mathbf{X})$ où $\mathcal{F}$ est une fonction dépendant des entrées, des poids et des biais.

Notations

- on note $w_{jk}^{[\ell]}$ les poids permettant d'aller vers la couche ℓ depuis le neurone k vers le neurone j ;
- $b_j^{[\ell]}$ le biais permettant d'aller sur le neurone j de la couche ℓ ;
- $f^{[\ell]}$ la fonction d'activation de la couche ℓ ;
- $n^{[\ell]}$ le nombre de neurones de la couche ℓ.

Définition *Équation de propagation*

Pour chacun des neurones $a_j^{[\ell]}$ on peut donc écrire l'équation de propagation qui lui est associé :

$$a_j^{[\ell]} = f^{[\ell]}\left(\sum_{k=0}^{n^{[\ell-1]}}\left(w_{jk}^{[\ell]}a_k^{[\ell-1]}\right) + b_j^{[\ell]}\right) = f^{[\ell]}\left(z_j^{[\ell]}\right)$$

Exemple :

Prenons un réseau de neurones à 3 couches (voir FIGURE 15) :

- 1 couche d'entrée à 2 neurones ;
- 1 couche cachée à 2 neurones, de fonction d'activation f_1 ;
- 1 couche de sortie à 1 neurone, de fonction d'activation f_2 ;

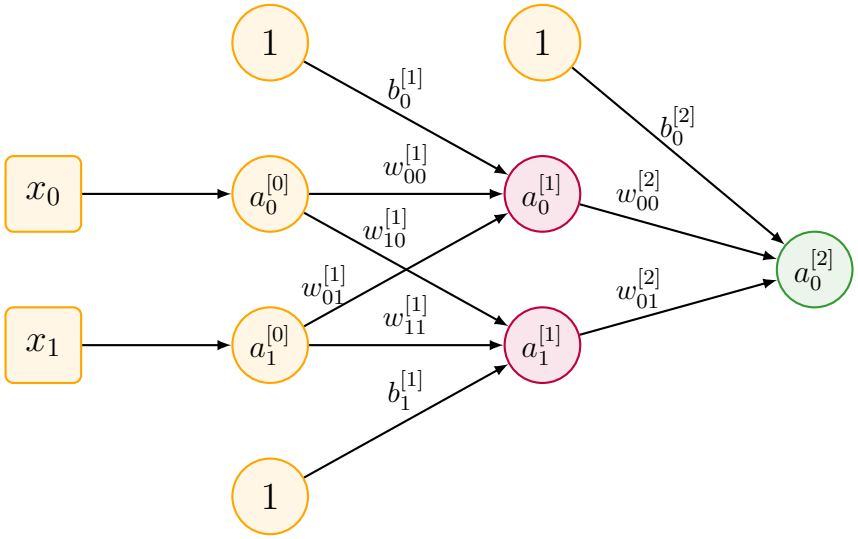


FIGURE 15 – Exemple de réseau de neurones à 3 couches

Initialisation les poids et le biais avec des valeurs aléatoires : $w_0 = -0,3$, $w_1 = 0,8$ et $b = 0,2$.

Il est possible d'écrire que : $y_0 = a_0^{[2]} = f_2\left(b_0^{[2]} + w_{00}^{[2]} a_0^{[1]} + w_{01}^{[2]} a_1^{[1]}\right)$

Par ailleurs :

$$a_0^{[1]} = f_1\left(b_0^{[1]} + w_{00}^{[1]} a_0^{[0]} + w_{01}^{[1]} a_1^{[0]}\right) \quad \text{et} \quad a_1^{[1]} = f_1\left(b_1^{[1]} + w_{10}^{[1]} a_0^{[0]} + w_{11}^{[1]} a_1^{[0]}\right)$$

Au final, on a donc :

$$y_0 = a_0^{[1]} = f_2\left(b_0^{[2]} + w_{00}^{[2]}\left(f_1\left(b_0^{[1]} + w_{00}^{[1]} a_0^{[0]} + w_{01}^{[1]} a_1^{[0]}\right)\right) + w_{01}^{[2]}\left(f_1\left(b_1^{[1]} + w_{10}^{[1]} a_0^{[0]} + w_{11}^{[1]} a_1^{[0]}\right)\right)\right)$$

Définition *Paramètres*

Les paramètres du réseau de neurones sont les poids et les biais, autant de valeurs que l'entraînement devra déterminer.

Méthode *Calcul du nombre de paramètres*

Soit un jeu de données étiquetées avec n entrées et p sorties.

On construit un réseau possédant ℓ couches et a_ℓ le nombre de neurones de la couche ℓ. Dans ce cas, la première couche est la couche d'entrée ($a_1 = n$) et la dernière couche et la couche de sortie ($a_\ell = p$).

Nombre de poids : $n_w = \sum_{i=1}^{\ell-1} (a_i \times a_{i+1})$.

Nombre de bais : $n_b = \sum_{i=2}^{\ell} (a_i)$.

Au final, le nombre total de paramètre à calculer est donné par $N = n_w + n_b$.

Objectif

Soit un jeu de données étiquetées. On note $\mathbf{X}$ le vecteur des données d'entrées. On note $\mathbf{Y}$ le vecteur des données de sorties. On note $\tilde{\mathbf{Y}}$ le vecteur de sortie calculé par le réseau de neurones.

L'objectif de la phase d'apprentissage du réseau de neurones est de déterminer les valeurs de l'ensemble des poids et des biais de telle sorte que l'écart entre $\mathbf{Y}$ et $\tilde{\mathbf{Y}}$ soit minimal.

4.3.2 Fonction de coût

Dans le but de minimiser l'écart entre la sortie du réseau de neurones et la valeur réelle de la sortie, on utilise une fonction de coût (ou fonction de perte). Il est possible de définir plusieurs types de fonctions, notamment en fonction du type de problème à traiter (classification ou régression par exemple).

Définition *Fonction coût régression*

Notons nb le nombre de données dans la base d'entraînement. Dans le cadre d'un problème de régression, on peut définir la fonction coût comme la moyenne des erreurs quadratique entre la valeur donnée par l'équation de propagation et la valeur de l'étiquette :

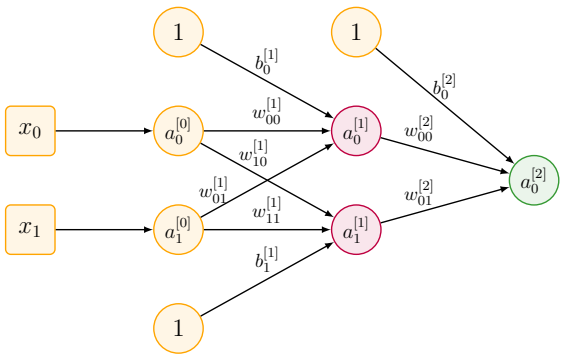
$$C = \frac{1}{nb}\sum_{i=1}^{nb}\left(\tilde{\mathbf{Y}}_\mathbf{i} - \mathbf{Y}_\mathbf{i}\right)^2$$

Objectif

L'objectif est dès lors de déterminer les poids et les biais qui minimisent la fonction coût.

4.3.3 Notion de rétropropagation – Descente de gradient

En réutilisant l'exemple de la FIGURE 15, nous allons présenter succinctement comment est minimisée la fonction coût. Pour cela, il va falloir dériver la fonction coût par rapport à chacune des variables.



Cherchons uniquement à déterminer le coût par rapport à un seul vecteur d'entrée du jeu d'entraînement. On a alors :

- $C = \left(\tilde{\mathbf{Y}}_\mathbf{i} - \mathbf{Y}_\mathbf{i}\right)^2 = \left(a_0^{[2]} - y\right)^2$;
- $a_0^{[2]} = f^{[2]}\left(z_0^{[2]}\right)$;
- $z_0^{[2]} = \sum_{k=0}^{1}\left(w_{0k}^{[2]} a_k^{[1]}\right) + b_0^{[2]}$.

Commençons par déterminer la dérivée partielle par rapport à un poids de la couche de sortie :

$$\frac{\partial C}{\partial w_{00}^{[2]}} = \frac{\partial C}{\partial a_0^{[2]}}\frac{\partial a_0^{[2]}}{\partial z_0^{[2]}}\frac{\partial z_0^{[2]}}{\partial w_{00}^{[2]}}$$

De même, on peut calculer la dérivée partielle du coût par rapport au biais : $\dfrac{\partial C}{\partial b_0^{[2]}} = \dfrac{\partial C}{\partial a_0^{[2]}} \dfrac{\partial a_0^{[2]}}{\partial z_0^{[2]}} \dfrac{\partial z_0^{[2]}}{\partial b_0^{[2]}}$

Calculons les dérivées nécessaires :

$$\frac{\partial C}{\partial a_0^{[2]}} = 2\left(a_0^{[2]} - y\right) \qquad \frac{\partial a_0^{[2]}}{\partial z_0^{[2]}} = f'^{[2]}\left(z_0^{[2]}\right) \qquad \frac{\partial z_0^{[2]}}{\partial w_{00}^{[2]}} = a_0^{[1]} \qquad \frac{\partial z_0^{[2]}}{\partial b_0^{[2]}} = 1$$

On a donc :

$$\frac{\partial C}{\partial w_{00}^{[2]}} = 2\left(a_0^{[2]} - y\right) f'^{[2]}\left(z_0^{[2]}\right) a_0^{[1]} \qquad \text{et} \qquad \frac{\partial C}{\partial b_0^{[2]}} = 2\left(a_0^{[2]} - y\right) f'^{[2]}\left(z_0^{[2]}\right)$$

Prenons le cas ou la fonction d'activation est la fonction identité. On a alors :

$$\frac{\partial C}{\partial w_{00}^{[2]}} = 2\left(a_0^{[2]} - y\right) a_0^{[1]} \qquad \text{et} \qquad \frac{\partial C}{\partial b_0^{[2]}} = 2\left(a_0^{[2]} - y\right) \dots$$

On va ainsi pouvoir exprimer $\frac{\partial C}{\partial w_{00}^{[2]}}$, $\frac{\partial C}{\partial w_{01}^{[2]}}$, $\frac{\partial C}{\partial b_0^{[2]}}$, ... On pourrait ici écrire 9 équations en fonction des différents poids, des biais et des entrées x_0 et x_1.

À partir de cela, on va modifier les poids comme suit :

$$w_{00,i+1}^{[2]} = w_{00,i}^{[2]} - \eta \frac{\partial C}{\partial w_{00,i}^{[2]}} \qquad \text{et} \qquad b_{0,i+1}^{[2]} = b_{0,i}^{[2]} - \eta \frac{\partial C}{\partial b_{0,i}^{[2]}}$$

On réitère ensuite les opérations précédentes jusqu'à réduction suffisante de la fonction coût.

Définition *Taux d'apprentissage*

On définit l'hyperparamètre $\eta \in [0, 1[$ comme étant le taux d'apprentissage. Si ce taux d'apprentissage est très grand, l'algorithme d'apprentissage mettra beaucoup de temps à trouver le minimum. S'il est trop grand, le minimum peut ne jamais être trouvé.

Définition *Gradient*

Dés lors, dans le cas présenté, on peut définir le gradient comme le vecteur :

$$\operatorname{grad} C = \begin{bmatrix} \dfrac{\partial C}{w^{[1]}} \\ \dfrac{\partial C}{b^{[1]}} \\ \vdots \\ \dfrac{\partial C}{w^{[l]}} \\ \dfrac{\partial C}{b^{[l]}} \end{bmatrix}$$

4.3.4 Fin d'apprentissage

Définition *Epoch*

On appelle *epoch* un cycle d'apprentissage où tous les poids et tous les biais ont été mis à jour en faisant passer toutes les données du jeu d'entraînement dans les algorithmes de propagation et de rétropropagation.

Les méthodes pour stopper l'apprentissage sont essentiellement empiriques. On pourrait en effet fixer un nombre d'epoch ou une valeur d'erreur admissible et s'arrêter à ce moment là. Dans la pratique, se focaliser sur l'erreur n'est généralement pas satisfaisant. En effet, il y a risque de « surapprentissage ».

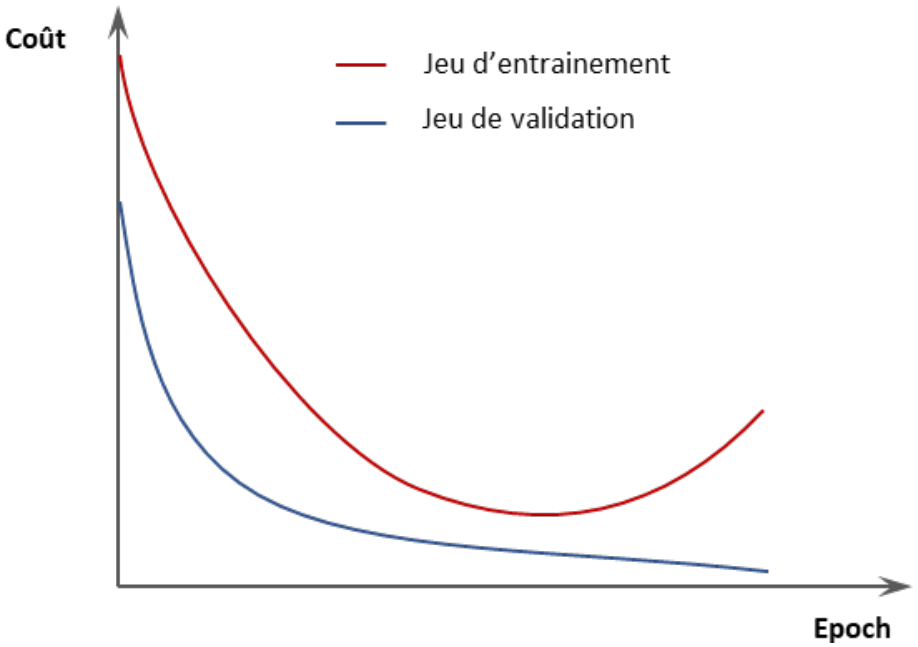


FIGURE 16 – Illustration du surapprentissage

Dans la figure suivante, on réalise un entraînement sur le jeu de données (une epoch) à la fin de l'epoch on dispose d'un premier modèle de réseau de neurones. On détermine alors l'erreur commise en utilisant le jeu d'entraînement puis l'erreur commise sur le jeu de validation. On calcule ensuite l'erreur commise par le modèle sur le jeu de validation. (On rappelle que le jeu de validation ne sert pas à modifier les poids et les biais.)

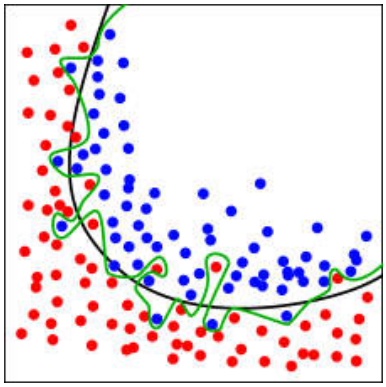
On réalise de même à la fin de l'epoch suivante etc...

« Logiquement » l'erreur décroît toujours avec le jeu d'entraînement car la descente du gradient a pour objectif de réduire cette erreur.

Sur le jeu de validation, l'erreur décroît pendant un certain nombre d'epoch puis augmente.

Il existe en fait un stade à partir duquel le réseau de neurones se spécialise sur le jeu d'entraînement et devient donc incapable de réaliser des prédictions fiables sur un nouveau jeu de données. On parle de surentraînement (ou d'overfitting).

Exemple :



La ligne verte représente un modèle sur-appris et la ligne noire représente un modèle régulier.

La ligne verte classifie trop parfaitement les données d'entraînement, elle généralise mal et donnera de mauvaises prévisions futures avec de nouvelles données.

Le modèle vert est donc au final moins bon que le noir.

4.4 Pour aller plus loin...

Les réseaux présentés ci-dessus sont les réseaux dits denses (ou fully-connected). On utilise d'autre types de réseaux pour une meilleure prédiction en fonction des données d'entrées :

- réseaux convolutifs (CNN) pour l'analyse d'images ;
- réseaux de neurones récurrents pour les données temporelles *etc.*.

Références

[1] A. Meurdefroid et X. Pessoles : Cours de sciences de l'ingénieur, 2021. PT, PSI*, MP - Lycée La Martinière Monplaisir - Lyon.

[2] E. Biernat et M. Lutz : *Data science : fondamentaux et études de cas.* Eyrolles, 2015.